DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Intelligent CPU Scheduler Simulator

Project Report

Operating Systems · CPU Scheduling Algorithms · Simulation & Visualization

Subject	Operating Systems
Topic	CPU Scheduling Algorithms
Interfaces	Command-Line Interface · Web-Based Interface
Algorithms	FCFS · SJF · Round Robin · Priority Scheduling

Table of Contents

1. Project Overview
2. Module-Wise Breakdown
3. Scheduling Algorithms
4. Performance Metrics
5. Technologies Used
6. System Flow Diagram
7. Revision Tracking on GitHub
8. Conclusion and Future Scope
9. References

1. Project Overview

The **Intelligent CPU Scheduler Simulator** is a software-based simulation tool developed to demonstrate and analyse the principal CPU scheduling algorithms employed in modern operating systems. CPU scheduling is central to process management, governing execution order and directly influencing system throughput, response time, and resource utilisation.

Users supply process parameters — arrival time, burst time, and priority value — whereupon the system produces a complete execution timeline (Gantt chart), per-process waiting and turnaround times, and their aggregated averages. The simulator is delivered via two interfaces: a command-line interface (CLI) for scripted or batch operation, and an interactive web-based interface featuring dynamic chart rendering.

Primary Objectives

- Provide a clear, interactive understanding of CPU scheduling concepts.
- Enable quantitative comparison of algorithm performance through computed metrics.
- Visualise process execution through dynamically generated Gantt charts.
- Support both command-line and web-based interaction paradigms.

2. Module-Wise Breakdown

The simulator is structured into five cohesive functional modules, each with a clearly defined responsibility. This modular architecture promotes maintainability, extensibility, and independent testability.

2.1 Input Module

Responsible for collecting process parameters from the user. Each process is assigned an auto-generated Process ID; the user provides Arrival Time (AT), Burst Time (BT), and Priority. Web inputs are captured via HTML form fields; CLI inputs are read interactively from the terminal.

2.2 Scheduling Module

The core computational module, implementing four scheduling algorithms. **FCFS** dispatches processes strictly in arrival order. **SJF** selects the process with the smallest remaining burst time. **Round Robin** allocates a fixed time quantum to each process in cyclical order. **Priority Scheduling** orders processes by assigned priority (lower value denotes higher priority).

2.3 Calculation Module

Derives three key performance indicators per process: Completion Time (CT), Turnaround Time (TAT = CT – AT), and Waiting Time (WT = TAT – BT). Aggregate averages for TAT and WT are computed to facilitate cross-algorithm benchmarking.

2.4 Visualisation Module

Generates Gantt charts depicting the process execution timeline, including idle CPU intervals. In the web version, charts are rendered dynamically using HTML, CSS, and JavaScript. In the Python version, visualisation is produced via *matplotlib*.

2.5 Output Module

Presents computed results in a structured format comprising a tabular process summary, per-process metrics, aggregated averages, and the Gantt chart. Output is delivered through both the CLI and the web interface.

3. Scheduling Algorithms

The table below summarises the four scheduling algorithms implemented within the simulator, their classification, primary selection criterion, and representative use cases.

Algorithm	Classification	Selection Criterion	Typical Use Case
FCFS	Non-preemptive	Order of arrival	Batch processing systems
SJF	Non-preemptive	Shortest burst time	Throughput optimisation
Round Robin	Preemptive	Fixed time quantum	Time-sharing systems
Priority Sched.	Non-preemptive	Priority value	Real-time systems

4. Performance Metrics

The Calculation Module evaluates the following performance indicators for every simulated process. These metrics provide a quantitative basis for comparing algorithm efficiency.

Metric	Formula	Description
Completion Time (CT)	—	Clock time at which the process finishes execution
Turnaround Time (TAT)	CT – AT	Total elapsed time from arrival to completion
Waiting Time (WT)	TAT – BT	Time the process spent idle in the ready queue
Avg. Turnaround Time	Σ TAT / n	Mean turnaround across all scheduled processes
Avg. Waiting Time	Σ WT / n	Mean waiting time across all scheduled processes

5. Technologies Used

Category	Technology / Tool	Purpose
Language	Python	Backend logic and CLI simulation
Language	JavaScript	Web-based simulation and DOM interaction
Language	HTML / CSS	Web interface structure and styling
Library	matplotlib	Gantt chart plotting within Python
Library	collections.deque	Round Robin ready-queue implementation
Library	DOM Manipulation	Dynamic real-time updates in the browser
Tool	GitHub	Version control and project tracking
Tool	VS Code	Integrated development environment
Tool	Linux Terminal	Execution, testing and debugging

6. System Flow Diagram

The diagram below illustrates the complete end-to-end execution flow of the simulator, from initial user input through algorithm selection, data processing, Gantt chart generation, metric computation, and final output presentation.

Figure 1. End-to-End Execution Flow — CPU Scheduler Simulator

7. Revision Tracking on GitHub

Version control is maintained using **GitHub** under the repository **CPU-Scheduler-Simulator**. The following table documents the principal commit milestones recorded during development.

#	Commit Summary	Scope / Notes
1	Initial project setup	Directory structure, environment configuration
2	Implementation of FCFS	First-Come First-Served scheduling logic
3	Addition of SJF algorithm	Shortest Job First logic and unit tests
4	Implementation of Round Robin	Cyclic scheduling with configurable quantum
5	Priority Scheduling	Priority-based execution order
6	Gantt chart visualisation	matplotlib and JavaScript chart integration
7	UI improvements and bug fixes	Web interface refinement and stability patches

8. Conclusion and Future Scope

Conclusion

The Intelligent CPU Scheduler Simulator successfully replicates the behaviour of four fundamental scheduling algorithms and delivers both textual and graphical representations of process execution. The computed performance metrics — waiting time, turnaround time, and their respective averages — provide a rigorous basis for comparing algorithm efficiency under varied workload conditions. The dual-interface design (CLI and web) renders the system versatile and accessible for academic study and practical benchmarking alike.

Future Scope

The following enhancements have been identified as viable extensions to the current implementation:

Enhancement	Description
Preemptive Algorithms	Add SRTF (Shortest Remaining Time First) and Preemptive Priority scheduling
GUI Framework	Develop a desktop GUI using Tkinter or a web front-end using React.js
Real-Time Animation	Animate process movement across the ready queue in real time
Comparison Graphs	Side-by-side bar charts of WT/TAT across all algorithms
Export Functionality	Allow results to be exported to CSV, Excel, or PDF format
Online Deployment	Host the web application on a public cloud platform (Heroku / Vercel)

9. References

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons.

[2] GeeksforGeeks. (n.d.). *CPU Scheduling Algorithms*. Retrieved from https://www.geeksforgeeks.org/cpu-scheduling-in-operating-systems/

[3] Python Software Foundation. (2024). *Python 3 Official Documentation*. Retrieved from https://docs.python.org/3/

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

[5] Mozilla Developer Network. (2024). *MDN Web Docs: JavaScript, HTML, CSS*. Retrieved from https://developer.mozilla.org/